UNIVERSIDAD DE MANILA • COLLEGE OF COMPUTING STUDIES
Human Computer Interaction (ITE 323) • Prof. Mary Grace Nulud
bikko.studio
Academic Year 2026

Technical Documentation and Member Assignment: Gwyneth Vienne G. Villanueva

- **Member Name**: Gwyneth Vienne G. Villanueva (Gwyneth)
- **Assigned Role**: Role 3: Database and Queries (Cloud Database, Schema and Secure Storage Developer)
- **Project Title**: bikko.studio (System Architecture: studioNeo / sankobite-v3)
- **Institution**: Universidad de Manila, College of Computing Studies
- **Course**: Human Computer Interaction (ITE 323)

1. Role Overview and Scope of Work

Gwyneth serves as the Cloud Database, Schema, and Secure Storage Developer. Her scope of work encompasses relational database modeling (100% Boyce-Codd Normal Form compliance), Azure Database for MySQL Flexible Server provisioning, high-performance connection pooling, security credential storage, continuous feed and category query optimization, and time-gated Azure Blob Storage integration.

Her primary objective is to safeguard artist assets against unauthorized copying, hotlinking, and scraping while delivering instant, index-backed query responses for the social art feed and digital asset vault.

2. Core Technical Responsibilities

2.1. Cloud Database Setup and User Schemas

1. **Azure MySQL Flexible Server Provisioning (Specific Objective 2, Section 2.3.4)**:
2. Configure the managed Azure Database for MySQL Flexible Server using `utf8mb4` character set and `utf8mb4_unicode_ci` collation to support international creator handles and emoji reactions.
3. Enforce transport layer security using TLS 1.3 with official DigiCert Global Root CA certificates.
4. Implement high-performance connection pooling using `mysql2/promise` with configurable limits (`waitForConnections: true`, `connectionLimit: 10`, `queueLimit: 0`) to prevent connection exhaustion during traffic spikes.
5. Enforce parameterized queries using prepared statements (`?` placeholders) for 100% protection against SQL injection attacks.
6. Core files: `src/lib/db.ts`, `scripts/schema.sql`.
7. **Member Profile and Badge Schema (100% BCNF Compliant)**:
8. Design relational tables where every determinant is a superkey, eliminating data redundancy and update anomalies: `sql CREATE TABLE IF NOT EXISTS `accounts` ( `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT, `email` VARCHAR(255) NOT NULL, `handle` VARCHAR(64) NOT NULL, `display_name` VARCHAR(255) NOT NULL, `avatar_url` TEXT NULL DEFAULT NULL, `bio` TEXT NULL DEFAULT NULL, `website_url` VARCHAR(512) NULL DEFAULT NULL, `is_verified` TINYINT(1) NOT NULL DEFAULT 0, `system_role` ENUM('user', 'curator', 'admin') NOT NULL DEFAULT 'user', `artist_distinction` ENUM('none', 'artist', 'featured', 'partner') NOT NULL DEFAULT 'none', `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, `updated_at` TIMESTAMP NOT NULL DEFAULT

`CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, PRIMARY KEY (`id`), UNIQUE KEY `idx_accounts_email` (`email`), UNIQUE KEY `idx_accounts_handle` (`handle`), KEY `idx_accounts_is_verified` (`is_verified`), KEY `idx_accounts_artist_distinction` (`artist_distinction`) ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;`

9. Stores user profiles, Member IDs (#1042), and verification distinctions (Partner, Featured Artist, Partner Verified).
10. **Authentication Credential Storage**:
11. **WebAuthn Credentials Table (`webauthn_credentials`)**:
    - Stores hardware credential IDs, public keys (`public_key` BLOB/TEXT), monotonic signature counters to detect authenticator cloning, device transport types (internal, usb, nfc, ble), and backup eligibility flags.
12. **Magic Tokens Table (`auth_magic_tokens`)**:
    - Stores 6-digit email OTP hashes (`token_hash` VARCHAR(64)), issuance timestamps, and expiration cutoffs (10-minute validity window).
13. **TOTP Credentials Table (`totp_credentials`)**:
    - Stores encrypted shared secrets (`secret_ciphertext`), confirmation status, and initialization vectors.
14. **Backup Codes Table (`totp_backup_codes`)**:
    - Stores salted hashes of single-use emergency recovery keys with consumption timestamps (`used_at`).
15. **OAuth Providers Table (`account_oauth_providers`)**:
    - Maps third-party Google and Microsoft identities to internal account records.

2.2. Feed, Story and Asset Vault Queries

1. **Continuous Feed and Active Story Queries (FR4 - Curated Art Feed)**:
2. Write paginated SQL queries joining `artworks` and `accounts` to stream creative posts with creator metadata (handle, avatar, location, timestamps, format tags): `sql SELECT a.id, a.code, a.title, a.description, a.category, a.like_count, a.comment_count, a.created_at, acc.handle, acc.display_name, acc.avatar_url, acc.artist_distinction FROM artworks a INNER JOIN accounts acc ON a.account_id = acc.id WHERE a.status = 'published' ORDER BY a.created_at DESC LIMIT ? OFFSET ?;`
3. Active 24-hour Story query retrieving recent artwork reels published within the last day: `sql SELECT * FROM artworks WHERE created_at >= NOW() - INTERVAL 24 HOUR ORDER BY created_at DESC;`
4. **Category Filter Indexing**:
5. Optimize feed filter tabs (All Works, Partners, and Featured) by indexing `artist_distinction` and `category` :
    - All Works: Queries all published posts (`WHERE status = 'published'`).
    - Partners: Filters by `acc.artist_distinction = 'partner'` .
    - Featured: Filters by `acc.artist_distinction = 'featured'` .
6. Return dynamic count headers for UI badge chips (`ALL WORKS (12)` , `PARTNERS (5)`).
7. **Engagement Metrics and Transactional Integrity**:

8. **Double-Tap Like Counter**: Use the `artwork_likes` table with a composite primary key `(account_id, artwork_id)` to enforce unique likes per user. Execute an atomic transaction to insert the like record and update the cached `like_count` in the `artworks` table: `sql START TRANSACTION; INSERT IGNORE INTO artwork_likes (account_id, artwork_id) VALUES (?, ?); UPDATE artworks SET like_count = (SELECT COUNT(*) FROM artwork_likes WHERE artwork_id = ?) WHERE id = ?; COMMIT;`
9. Track comment totals, user bookmarks, and download counts (`artwork_downloads` table) for artist analytics.
10. **Asset Vault Catalog Queries (FR6 - Multi-Format Asset Storage Vault)**:
11. Structure database queries for the Digital Asset Vault filtering across All Categories, Vector Stems, Color Studies, and Typography: `sql SELECT aa.id, aa.asset_type, aa.file_size_bytes, aa.mime_type, a.title, a.code, acc.handle FROM artwork_assets aa INNER JOIN artworks a ON aa.artwork_id = a.id INNER JOIN accounts acc ON a.account_id = acc.id WHERE (? IS NULL OR a.category = ?) ORDER BY a.created_at DESC;`

2.3. Cloud Blob Storage and Time-Gated Download Links

1. **Membership Permission Checks (Specific Objective 2, FR7 - Time-Limited Secure Download Links)**:
2. Before any master vector or high-resolution file is provided, query the database to verify active subscription or verified partner standing: `sql SELECT id, status, expires_at FROM subscribers WHERE account_id = ? AND status = 'active' AND expires_at > NOW();`
3. Deny request if subscriber status is inactive, expired, or unverified.
4. **15-Minute Temporary Download Links (Shared Access Signatures)**:
5. Integrate Azure Blob Storage using `@azure/storage-blob` (`src/lib/azure-storage.ts`).
6. Generate secure Shared Access Signature (SAS) URLs valid for exactly 15 minutes (`AZURE_STORAGE_SAS_EXPIRY_MINUTES = 15`).
7. Apply strict read-only permissions (`BlobSASPermissions.parse("r")`).
8. Attach clean download headers to prompt immediate browser saving rather than direct viewing: `contentDisposition: 'attachment; filename="studioNeo-artwork-master.svg"'` .
9. Benefits:
    - Eliminates permanent public URLs.
    - Prevents automated bots and web crawlers from scraping high-resolution art.
    - Blocks hotlinking and unauthorized link forwarding across the web.

---

3. Documentation Sections Owned

Gwyneth is directly responsible for drafting, updating, and maintaining the following formal sections in the project report:

1. **Section 1.2.2: Specific Objective 2 (Secure Cloud File Sharing System)**:
2. Define the database-backed permission checking mechanism and encrypted cloud storage architecture designed to stop art theft and direct file link scraping.
3. **Section 2.2.3: Asset Integrity and Art Protection**:
4. Detail user needs surrounding creative asset ownership, protection against unauthorized AI scrapers, guaranteed file safety, and reliable downloads for paid members.
5. **Section 2.3.1: Functional Requirements (FR4, FR5, FR6, FR7)**:

6. FR4: Curated Art Feed (Paginated feeds, 24-hour Story reels, category filter indexes, like transactions).
7. FR5: Desktop Profile Sidebar (Database queries retrieving user statistics, member IDs, badges, and security status).
8. FR6: Multi-Format Asset Storage Vault (Catalog schema for Vector Stems, Color Studies, and Typography).
9. FR7: Time-Limited Secure Download Links (Entitlement validation, 15-minute temporary read-only SAS generation).
10. **Section 3.2.3: Storyboard Scenario 3 (Asset Discovery, Multi-Format Conversion and Time-Gated Download)**:
11. Document the step-by-step user interaction flow from gallery inspection to database permission verification, SAS token generation, and secure file download.

---

4. Cross-Role Integration and Development Handoffs

- **Handoff to Role 1 (Jaz - Frontend UI)**:
- Gwyneth delivers optimized SQL query results and JSON endpoints feeding Jaz's responsive feed cards, filter chips, and profile sidebar.
- When a member clicks download in Jaz's export popup modal, Gwyneth runs the entitlement check and generates the 15-minute temporary link.
- **Handoff to Role 2 (Adrian - System Framework)**:
- Gwyneth establishes the connection pool and provides database helper functions (`query()` , `getAccountById()` , `saveWebAuthnCredential()`) to Adrian's API routes.
- Adrian's authentication middleware validates session cookies and passes the authenticated `accountId` to Gwyneth's query functions.
- **Handoff to Role 4 (Paisley - Research and QA)**:
- Paisley provides artist seed data (profile bios, handles, badges, showcase artwork descriptions) for Gwyneth to populate into MySQL tables.
- Paisley tests the 15-minute expiration window to ensure download links expire on schedule and verify that unauthorized users are blocked.
